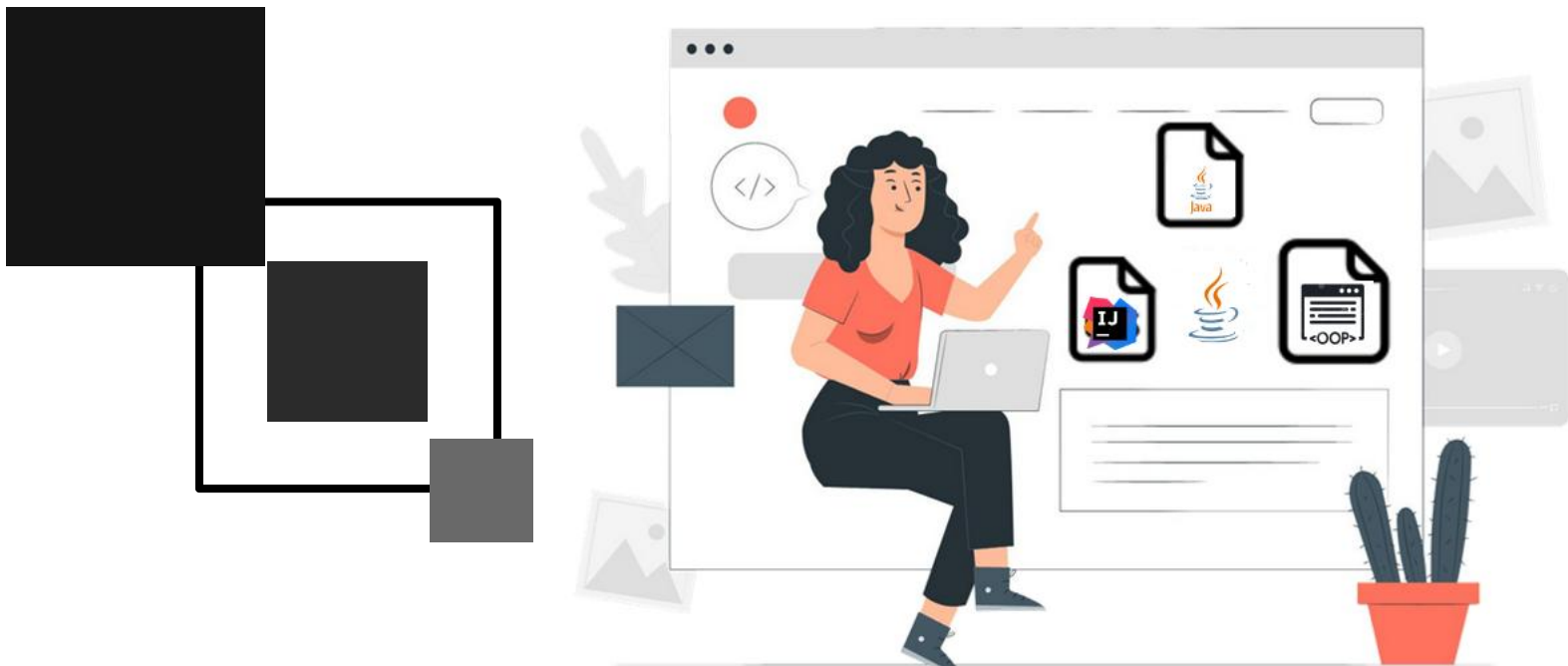




Domain Driven Design

Eliane Marion

FIAP

FERRAMENTAS DE TRABALHO



JAVA

Linguagem de programação orientada a objetos e amplamente utilizada



INTELLIJ

Ambiente de desenvolvimento integrado (IDE) usado na programação de computadores

MÉTODOS AVALIATIVOS



CHECKPOINTS

Exercícios ou projetos
práticos desenvolvidos
em laboratório.



Global Solution

Avaliação prática
abordando assuntos
trabalhados em aula.



Challenge

Entrega bimestral do
projeto seguindo
orientações.



01

HERANÇA
Conceitos básicos

HERANÇA

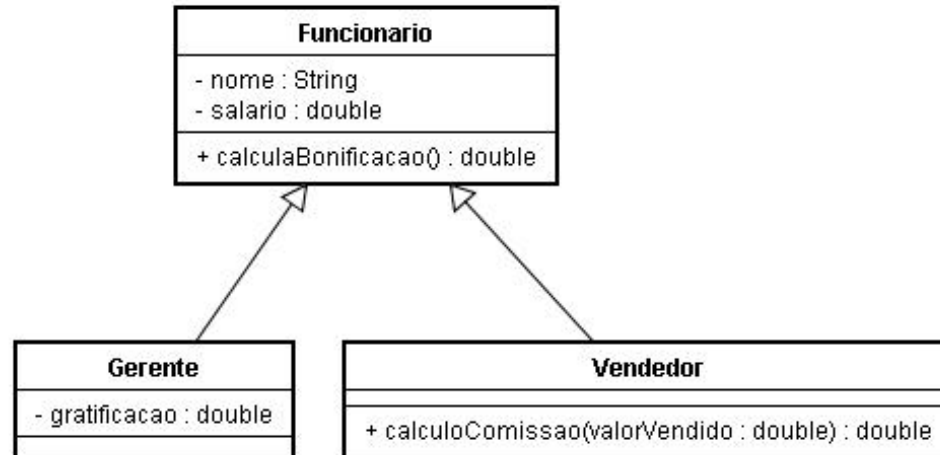
- É um princípio da Programação Orientada a Objetos que permite que as classes compartilhem atributos e métodos comuns baseados em um relacionamento.
- Possibilita a aplicação de vários conceitos de orientação a objetos que não seriam viáveis sem a organização e estruturação alcançada por sua utilização.
- A habilidade de estender é considerada um dos maiores benefícios da orientação a objetos.
- A classe original é conhecida como **superclasse**

HERANÇA

- Quando você estende uma classe para criar uma nova classe, uma **subclasse** a nova classe estendida **herda** campos e métodos da superclasse.

Superclasse

- É também conhecida como “classe mãe” e a subclasse como “classe filha”.



HERANÇA

Generalização

- A generalização é o agrupamento de características (atributos) e regras (métodos) comuns em um modelo de sistema, então a **superclasse** contém o que é **genérico** da classe.
- A classe Funcionario contém dois atributos e um método, os quais são comuns para gerentes e vendedores.



HERANÇA

Especialização

- A especialização é o processo inverso, é a definição das **particularidades** de cada elemento de um modelo de sistemas, detalhando características e regras específicas de um o objeto, então a subclasse contém o que é **específico** da classe.
- O gerente ganha gratificação e o vendedor ganha comissão das vendas, mas ambos tem nome, salario e ganham bonificação.



HERANÇA EM JAVA

Em Java, é utilizada a cláusula **extends** na criação da classe (para mostrar que a classe Gerente herda atributos e métodos da classe Funcionario).

Podemos dizer que o objeto do tipo Gerente estende os atributos e métodos do objeto do tipo Funcionario, pois agora Gerente é um Funcionario.

HERANÇA NA PRÁTICA

```
public class Gerente extends Funcionario;  
{  
    public double classificacao;  
}
```



HERANÇA EM JAVA

Vantagens

Evitar duplicação de código - Não reescrever o código

Reutilização de código - Código existente pode ser reutilizado Se já existir uma classe semelhante à quella da qual precisamos, às vezes podemos dividir a classe existente e reutilizar algum código existente, em vez de implementar tudo novamente, só tomando cuidado com herança imprópria.

HERANÇA EM JAVA

Desvantagens

Acoplamento - Em Java não existe um limite para uso da herança, porém um sistema com muita herança deixa o código acoplado

Quebra do encapsulamento E se precisamos acessar os atributos que herdamos? Não gostaríamos de deixar os atributos de Funcionario public pois dessa maneira qualquer um poderia alterar os atributos dos objetos deste tipo Existe outro modificador de acesso, o protected que fica entre o private e o public Um atributo **protected** só pode ser acessado (visível) pela própria classe ou suas subclasses e até pelas classes do mesmo pacote.

HERANÇA EM JAVA

Desvantagens

Uma classe pode ter várias filhas, mas pode ter apenas uma mãe.

SOBRECARGA E SOBRESCRITA

Sobrecarga

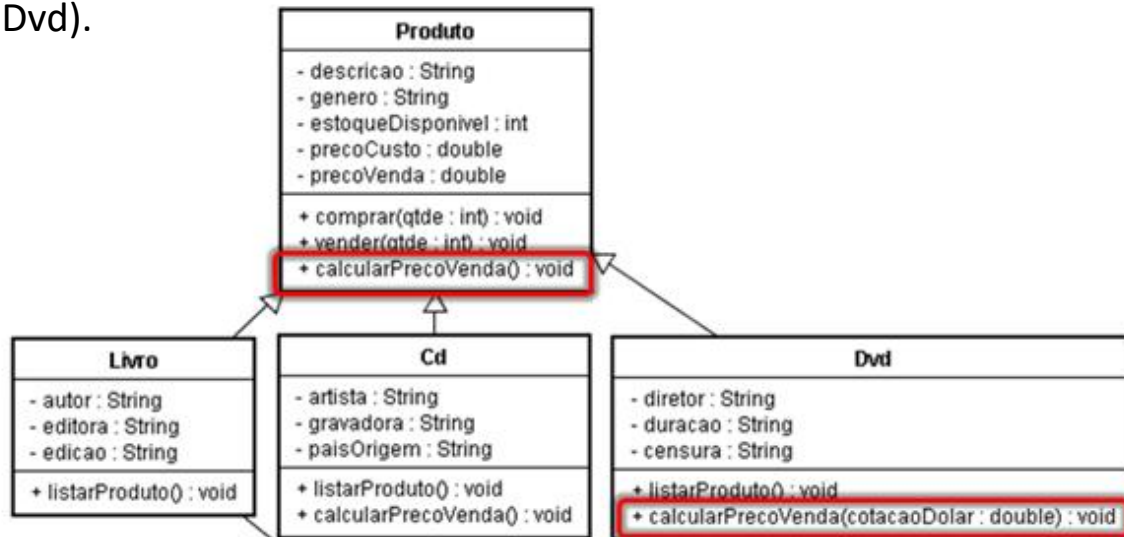
Sobrecarregar um método é fornecer mais de um método com o mesmo nome, mas com diferentes assinaturas para distinguí-los. Sobrecarregar um método herdado simplesmente significa que você adicionou um novo método, com o mesmo nome, mas com uma assinatura diferente, de um método herdado. O mesmo conceito aplicado em construtores.

Sobrescrita

Sobrescrever um método significa substituir a implementação da superclasse daquele método com a sua própria. A sobrescrita ou reescrita de método está diretamente relacionada com herança e é a possibilidade de manter a mesma assinatura de um método herdado e reescrevê-lo na subclasse.

SOBRECARGA E SOBRESCRITA

Exemplo de sobrecarga: Ressaltando que um objeto do tipo Dvd possui dois métodos calcularPrecoVenda, um sem parâmetro (herdado de Produto) e outro recebendo a cotação do dólar (sobrecarregado na classe Dvd).



SOBRECARGA E SOBRESCRITA

Exemplo de sobrescrita: Todo fim de ano, os funcionários recebem uma bonificação. Os funcionários comuns recebem 30% do valor do salário e os gerentes 50%. Então o método calculaBonificacao da classe Gerente herdado da classe Funcionario não é adequado, a solução é a reescrita de método.

```
public class Gerente extends Funcionario{  
  
    private double gratificacao;  
  
    // getters e setters omitidos  
  
    public double calculaBonificacao() {  
        return this.salario * 0.5;  
    }  
}
```


SOBRECARGA E SOBRESCRITA

Na chamada de um método sobrescrito o Java considera primeiro a classe a partir da qual o objeto foi instanciado, se a superclasse possuir um método com a mesma assinatura este será descartado.

Depois de reescrito, não podemos mais chamar o método antigo que fora herdado da superclasse, realmente alteramos o seu comportamento Mas podemos invocá-lo no caso de estarmos dentro da classe.

Uma invocação de super método sempre usa a implementação do método que a superclasse define (herda).

SOBRECARGA E SOBRESCRITA

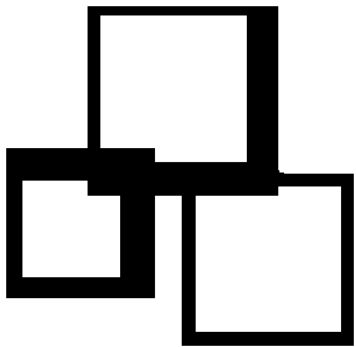
Exemplo: Imagine que para calcular a bonificação de um Gerente devemos fazer igual ao cálculo de um Funcionario porem adicionando R\$ 1000. Poderíamos fazer assim: O método calculaBonificacao da classe Funcionario continua calculando 30% do valor do salário para todos os funcionários.

```
public class Funcionario {  
  
    private String nome;  
    private double salario;  
  
    // getters e setters omitidos  
  
    public double calculaBonificacao() {  
        return this.salario * 0.3;  
    }  
}
```

SOBRECARGA E SOBRESCRITA

O método calculaBonificacao da classe Gerente é reescrito adicionando R\$ 1000.

```
public class Gerente extends Funcionario{  
  
    private double gratificacao;  
  
    // getters e setters omitidos  
  
    public double calculaBonificacao() {  
        return super.calculaBonificacao() + 1000;  
    }  
}
```



OBRIGADO

To be continued...

